1. 과제 개요

1.1 연구 배경 및 필요성

CNN은 영상 인식 분야에서 널리 활용되지만, 추론 과정에서 반복되는 convolution 연산과 intermediate activation의 데이터 이동으로 인해 높은 연산량과 에너지 비용이 발생한다. 특히 CNN 가속기에서는 산술 연산뿐만 아니라 온칩 및 오프칩 메모리 사이의 데이터 이동을 줄이는 것이 전체 에너지 효율을 결정하는 중요한 요소이다 [1].

Winograd 최소 필터링 알고리즘은 작은 크기의 convolution을 타일 단위로 변환하여 direct convolution보다 적은 수의 곱셈으로 계산할 수 있다 [2]. 그러나 실제 하드웨어에서는 입력 및 출력 변환을 위한 추가 연산과 변환 도메인의 중간 데이터 관리가 필요하므로, 단순한 곱셈 횟수 감소만으로 전체 효율을 보장하기 어렵다.

또한 convolution 결과를 메모리에 저장한 뒤 후속 연산을 별도로 수행하면 불필요한 intermediate activation 이동이 발생한다. 이러한 비용은 인접 연산을 하나의 datapath에서 연속적으로 처리하는 operation fusion을 통해 줄일 수 있다 [3]. 따라서 Winograd 알고리즘의 장점을 실제 하드웨어 효율로 연결하려면 연산기뿐만 아니라 타일 생성, 중간 버퍼링 및 후속 연산을 포함한 전체 데이터 흐름을 함께 고려해야 한다.

1.2 과제 개요

본 과제에서는 3x3 convolution을 대상으로 하는 **Winograd CNN Accelerator with Tile-Level Operation Fusion**을 설계한다. 제안 가속기는 Winograd 변환을 통해 convolution의 곱셈 횟수를 줄이고, 생성된 출력 타일에 후속 연산을 연속적으로 적용하여 중간 결과의 저장과 데이터 이동을 최소화하는 것을 목표로 한다.

또한 제한된 연산 자원을 CNN의 여러 계층에서 재사용하고, 계층별 activation의 사용 시점을 고려한 local storage를 적용한다. 이를 통해 개별 연산의 처리량만 높이는 구조가 아니라, 제한된 하드웨어 자원으로 CNN 추론 전체를 수행할 수 있는 합성 가능한 ASIC 가속기를 구현하고자 한다.

본 과제의 주요 수행 범위는 다음과 같다.

- Winograd 기반 CNN 모델 및 정수 연산 환경 구성
- 타일 단위 convolution과 후속 연산을 결합한 가속기 설계
- 합성 가능한 RTL 구현과 기능 시뮬레이션
- 논리 합성을 통한 동작 주파수, 면적 및 전력 분석

2. 설계 목표

본 과제는 baseline CNN을 Winograd 연산에 적합한 형태로 재구성하고, 정수 연산 기반 CNN 가속기를 synthesizable Verilog RTL로 구현하는 것을 목표로 한다. 연산량뿐만 아니라 transform overhead, intermediate tensor 이동, activation 저장 공간 및 계층별 하드웨어 이용률을 함께 고려하여 전체 architecture와 dataflow를 설계하였다.

2.1 전체 아키텍처 및 설계 방향

제안한 Winograd CNN 가속기의 전체 구조는 Figure 1과 같다.

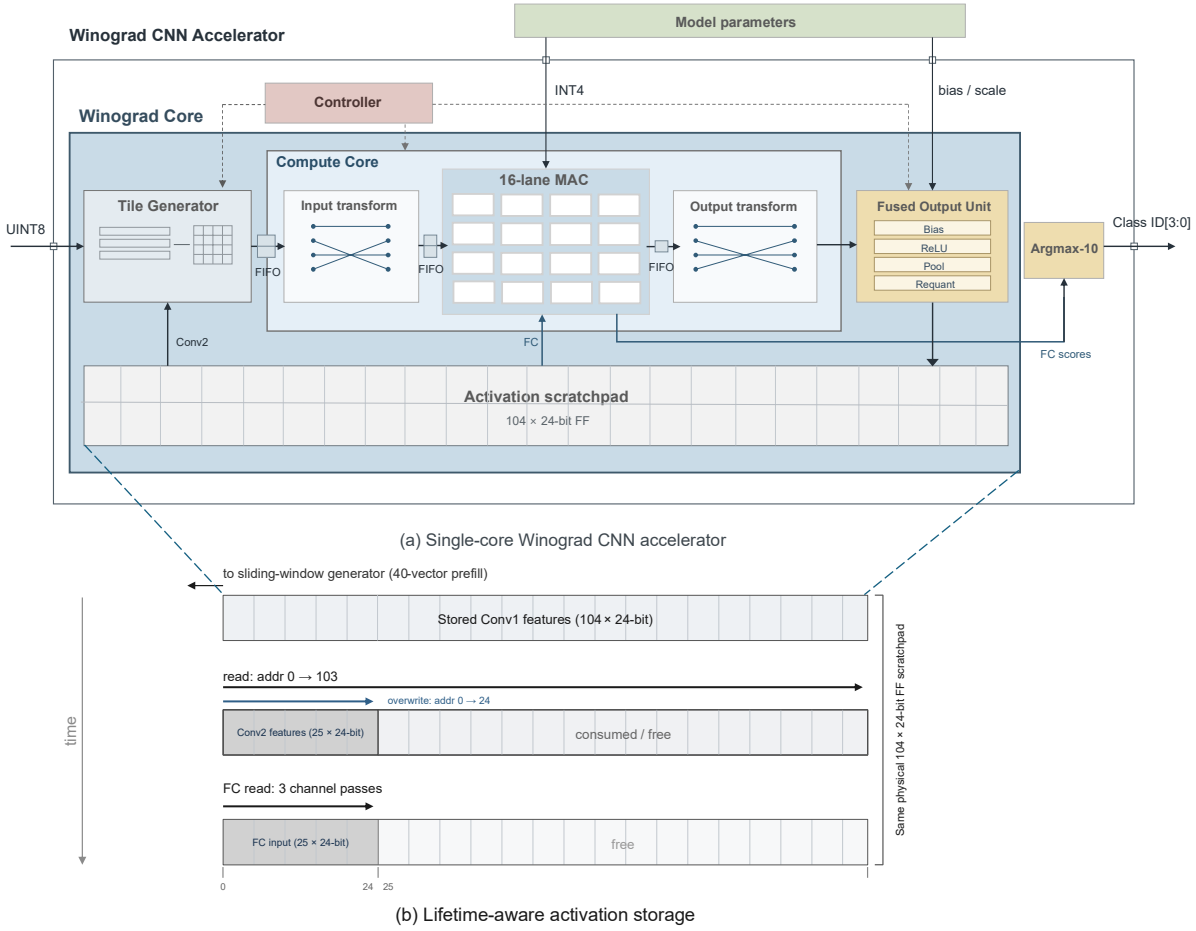


Figure 1. Proposed Winograd CNN accelerator:
(a) Single-core Winograd CNN accelerator architecture. (b) Lifetime-aware activation storage.

Figure 1(a)의 가속기는 Controller, Winograd Core 및 Argmax-10으로 구성된다. Model parameter는 가속기 외부의 interface를 통해 공급된다. Winograd Core 내부에는 Tile Generator, Compute Core, Fused Output Unit 및 Activation Scratchpad가 배치된다. 외부 UINT8 pixel stream은 Tile Generator로 입력된다. Tile Generator는 연속적으로 입력되는 pixel을 line buffer에 저장하고, convolution에 필요한 input tile을 생성한다. 생성된 tile은 FIFO를 통해 Compute Core로 전달되며, Compute Core는 input transform, 병렬 multiplication과 channel reduction, output transform을 수행한다.

Output transform 결과는 Fused Output Unit으로 직접 전달된다. 이 블록은 bias, ReLU, max-pooling 및 requantization을 연속적으로 수행하고, 다음 layer에서 사용할 UINT8 activation만 Activation Scratchpad에 저장한다.

Conv2에서는 scratchpad에 저장된 Conv1 activation을 Tile Generator가 다시 읽어 동일한 Winograd Core에서 처리한다. Conv2 연산이 완료되면 Compute Core의 MAC resource를 FC mode로 전환하여 재사용한다. FC 연산으로 생성된 10개의 score는 Argmax-10으로 전달되며, 최종적으로 4-bit Class ID를 출력한다.

Figure 1(b)는 Activation Scratchpad의 시간적 재사용 방식을 나타낸다. Conv1 activation이 Conv2에서 소비되면 해당 주소의 lifetime이 종료되며, Conv2 결과는 더 이상 사용되지 않는 주소부터 overwrite된다. 이후 FC layer는 동일한 물리적 저장 공간의 Conv2 결과를 입력으로 사용한다.

이 구조를 통해 본 설계는 하나의 Winograd Core와 Activation Scratchpad를 CNN의 여러 layer에서 반복적으로 사용한다. 각 블록의 내부 회로와 세부 pipeline 동작은 3절에서 설명한다.

2.2 Winograd Convolution 원리

일반적인 direct convolution은 kernel을 한 칸씩 이동시키며 각 출력 위치를 독립적으로 계산한다. 이에 비해 Winograd convolution은 서로 인접한 여러 출력을 하나의 tile로 묶어 계산함으로써 convolution에 필요한 multiplication 수를 줄이는 알고리즘이다 [2].

Winograd 알고리즘은 일반적으로 $F(m \times m,\ r \times r)$로 표기한다. 여기서 $\mathrm{r} \times \mathrm{r}$은 convolution kernel의 크기이며, $\mathrm{m} \times \mathrm{m}$은 한 번의 연산으로 생성되는 output tile의 크기이다. 연산에 필요한 input tile의 한 변 크기 $\boldsymbol{\alpha}$는 다음과 같다.

$$\alpha = m + r - 1 \tag{1}$$

본 설계에서 사용하는 $F(2 \times 2, 3 \times 3)$에서는 $m = 2,\ r = 3$이므로 4×4 input tile과 3×3 weight로부터 2×2 output tile을 생성한다.

Winograd convolution의 연산 과정은 다음과 같이 표현된다.

$$U = GgG^T \tag{2}$$

$$V = B^T dB \tag{3}$$

$$M_k = \sum_c U_{k,c} \odot V_c \tag{4}$$

$$Y_k = A^T M_k A \tag{5}$$

각 기호의 의미는 다음과 같다.

- d: 4×4 input activation tile
- g:3×3 spatial weight
- V: transform-domain activation
- U: transform-domain weight

- M: element-wise multiplication과 input-channel reduction 결과
- Y: 복원된 2×2 output tile
- A, B, G: 고정된 Winograd transform matrix
- $\odot$: element-wise multiplication

먼저 input activation과 weight를 각각 Winograd domain으로 변환한다. 변환 결과인 V와 U는 모두4×4크기이며, 같은 위치의 원소끼리 총 16회의 multiplication을 수행한다. 여러 input channel이 존재하면 channel별 $U \odot V$결과를 동일한 위치끼리 더하여 channel reduction을 수행한다. 마지막으로 output transform을 적용하여 2×2결과 Y를 복원한다.

Direct 3×3 convolution으로 2×2 출력을 계산하려면 input/output channel 조합마다 다음과 같이 36회의 multiplication이 필요하다.

반면 Winograd $F(2 \times 2, 3 \times 3)$에서는 4×4 transform domain의 element-wise multiplication만 필요하므로 이를 16회로 줄일 수 있다.

다만 multiplication 감소를 위해 input/output transform과 중간 데이터가 추가된다. 따라서 실제 하드웨어에서는 multiplication 수뿐만 아니라 transform datapath, register, buffer 및 데이터 이동을 함께 고려해야 한다. Weight transform은 inference 전에 수행할 수 있으므로, 본 가속기는 spatial weight g가 아니라 미리 변환 및 양자화된 U를 model parameter로 입력받는다. Bias, ReLU, pooling 및 requantization은 Winograd 수식에 포함되지 않으며, Y가 복원된 이후 Fused Output Unit에서 처리한다.

2.3 대상 CNN 구조 재구성

Baseline CNN은 두 개의 5×5 convolution layer와 두 개의 max-pooling layer, 하나의 FC layer로 구성된다. 본 설계에서는 전체 layer 순서와 channel 수는 유지하면서 convolution kernel을 5×5에서 3×3으로 변경하였다.

Layer	Baseline (5×5)	Proposed (3×3)
Input	1×28×28	1×28×28
Conv1 output	3×24×24	3×26×26
Pool1 output	3×12×12	3×13×13
Conv2 output	3×8×8	3×11×11
Pool2 output	3×4×4	3×5×5
FC input	48	75
FC output	10	10

Table 1. Comparison of the baseline and proposed CNN architectures.

5×5 kernel에도 Winograd 알고리즘을 적용할 수 있다. 그러나 5×5 kernel로 2×2 output tile을 생성하는 $F(2 \times 2,\ 5 \times 5)$ 에서는 식 (1)에 따라 input tile의 크기가 6×6으로 증가한다. 따라서 transform-domain element 수는 $F(2 \times 2,\ 3 \times 3)$의 16개에서 36개로 증가한다. 이는 multiplier 수뿐만 아니라 transform 연산기, 중간 register, buffer 및 datapath bit-width 증가로 이어진다.

본 과제는 제한된 하드웨어 자원으로 전체 CNN inference를 수행하는 단일 코어 구조를 목표로 한다. 이에 따라 multiplication 감소 효과와 transform overhead를 함께 고려하여 $F(2 \times 2,\ 3 \times 3)$을 선택하고, 이를 적용할 수 있도록 convolution kernel을 3×3으로 변경하였다.

Kernel 변경에 따라 convolution과 pooling 이후의 activation 크기도 달라진다. 마지막 pooling 결과는 3×4×4에서 3×5×5로 변경되며, FC 입력은 48개에서 75개로 증가한다. 이러한 모델 변경이 정확도에 미치는 영향은 PyTorch FP32 사전학습을 통해 먼저 확인하고, 이후 Winograd-aware QAT와 정수 연산 검증을 수행하였다.

보고서에서는 다음 두 효과를 구분한다.

- 5×5에서 3×3으로의 변경: CNN 구조 재구성에 따른 연산량 변화
- Direct 3×3에서 Winograd $F(2 \times 2,\ 3 \times 3)$으로의 변경: Winograd 적용에 따른 multiplication 감소

따라서 전체 연산량 감소를 Winograd만의 효과로 간주하지 않고, 모델 구조와 연산 알고리즘을 함께 변경한 결과로 평가한다.

2.4 핵심 설계 목표

2.4.1 연산 효율 및 자원 재사용

Winograd를 통해 3×3 convolution의 multiplication 수를 줄이고, transform-domain의 16개 위치를 병렬로 처리하는 16-lane MAC array를 구성한다.

Conv1과 Conv2는 하나의 Winograd Core를 순차적으로 재사용한다. FC layer에서는 별도의 대규모 연산기를 추가하지 않고 MAC array를 FC mode로 전환하여 multiplier와 accumulator를 재사용한다.

또한 transformed weight와 FC weight를 INT4로 제한하고, layer 사이에 저장되는 activation은 UINT8로 표현한다. Transform과 accumulation에는 각 연산 범위에 적합한 별도의 bit-width를 적용하여 정확도와 하드웨어 비용 사이의 균형을 확보한다.

2.4.2 Tile-Level Operation Fusion

Output transform은 2×2 output tile을 생성한다. 본 설계에서는 이 결과를 intermediate buffer에 저장하지 않고 bias, ReLU, max-pooling 및 requantization으로 직접 전달한다.

이를 통해 각 연산을 독립적으로 수행할 때 발생하는 intermediate tensor의 write와 read를 제거하고, 후속 layer에서 사용할 UINT8 activation만 Activation Scratchpad에 저장한다. 본 설계에서는 이러한 구조를 Tile-Level Operation Fusion으로 정의한다 [3].

2.4.3 Activation Storage 재사용

Conv1과 Conv2를 위한 독립적인 full-size activation buffer를 배치하지 않고, Conv1 activation의 lifetime이

종료된 주소에 Conv2 결과를 overwrite한다. FC layer도 동일한 물리적 저장 공간의 Conv2 결과를 입력으로 사용한다.

Activation Scratchpad의 한 word는 동일한 spatial 위치의 3개 UINT8 channel을 묶은 24-bit 데이터로 구성한다. 이를 통해 channel별 저장소를 별도로 구성하지 않고 Conv2의 multi-channel 입력과 FC 입력을 동일한 형식으로 관리한다 [1].

2.4.4 Elastic Pipeline 구성

Tile Generator, input transform, MAC/channel reduction 및 output transform은 서로 다른 latency와 데이터 생성 주기를 가진다. 각 블록 사이에는 valid-ready 기반의 Elastic FIFO를 배치하여 producer와 consumer 사이의 일시적인 속도 차이를 흡수한다.

Downstream block이 데이터를 받을 수 없는 경우 ready 신호를 이용해 backpressure를 전달하며, FIFO에 저장된 데이터와 valid 상태를 유지한다. 이를 통해 내부 stall이 발생해도 데이터 손실이나 연산 순서의 변경 없이 pipeline을 동작시킨다.

2.5 정량적 설계 사양

제안 가속기에 적용한 수치 형식과 구현 사양은 Table 2와 같다.

Item	Specification
Target application	MNIST classification
CNN structure	Conv–Pool–Conv–Pool–FC
Convolution	Winograd F(2×2, 3×3)
Input / output tile	4×4 / 2×2
MAC parallelism	16 lanes
External / stored activation	UINT8
Winograd weight U	INT4
Transform-domain activation V	INT11
Compute / accumulation	INT18
Bias storage	INT16
FC weight	INT4
Activation scratchpad	104×24-bit FF array
Target accuracy	≥97% on the 1,000-image competition set
Implementation / evaluation	Synthesizable Verilog / accuracy, cycle, area, Fmax, power

Table 2. Numerical formats and implementation specifications of the proposed accelerator.

외부에서 입력되거나 Activation Scratchpad에 저장되는 activation은 UINT8로 통일한다. Winograd input transform 결과인 V는 INT11, transformed weight U와 FC weight는 INT4로 표현한다. Multiplication 이후의

channel reduction, output transform 및 FC accumulation은 INT18 범위에서 수행하며, bias는 INT16으로 저장한 뒤 compute datapath에서 sign-extension한다.

Activation Scratchpad는 104×24-bit FF array로 구성한다. 하나의 24-bit word에는 동일한 spatial 위치의 3개 UINT8 channel이 저장되며, Conv1, Conv2 및 FC가 동일한 저장 공간을 순차적으로 재사용한다.

각 bit-width는 임의로 결정한 값이 아니라 PyTorch QAT, 정수 golden model 및 evaluation dataset의 중간 연산 범위 측정을 통해 선정하였다. 다음 절에서는 Winograd-aware 모델 최적화 과정, 정수 연산 검증 및 각 하드웨어 블록의 세부 구현을 설명한다.

3. 설계 및 시뮬레이션

본 절에서는 제안한 Winograd CNN accelerator의 software–hardware co-design 과정과 RTL microarchitecture를 설명한다. 먼저 PyTorch 환경에서 Winograd-aware QAT를 수행하여 low-bit model을 생성하고, integer golden model을 이용하여 RTL의 연산 순서와 numerical format을 결정하였다. 이후 각 hardware block의 구조와 simulation 및 synthesis 결과를 분석한다.

3.1 Winograd-aware 모델 최적화

3.1.1 FP32 모델 학습 및 Winograd 동치 검증

2절에서 정의한 3×3 CNN을 먼저 spatial-domain FP32 모델로 학습하였다. 이 단계에서는 일반적인 PyTorch convolution을 사용하며, 각 convolution layer의 학습 parameter는 3×3 spatial weight g이다. 최종 FP32 모델은 MNIST test set 10,000장에 대해 97.03%의 accuracy를 기록하였다.

학습된 spatial weight를 Winograd 모델에 그대로 적용하여 일반 convolution과 $F(2 \times 2,\ 3 \times 3)$ Winograd convolution의 연산 결과가 일치하는지 검증하였다. Winograd 모델에서는 매 forward 과정에서 spatial weight g를 다음과 같이 transform-domain weight U로 변환한다.

$$U = GgG^T \tag{6}$$

Activation은 4×4 input tile 단위로 변환되고, transform-domain의 element-wise multiplication과 channel reduction을 거친 후 2×2 output tile로 복원된다. 일반 convolution과 Winograd convolution의 output은 FP64 연산 기준 최대 절대 오차 10^{-12} 이내에서 일치하였다. 또한 전체 CNN의 output logit과 gradient 전달을 검증하여, spatial model을 Winograd 연산으로 대체해도 학습과 inference의 수학적 동치가 유지됨을 확인하였다.

Conv2의 output 크기는 11×11로 tile 크기 2의 배수가 아니다. 이에 따라 마지막 tile의 오른쪽 또는 아래쪽에 zero padding을 적용하여 Winograd 연산을 수행하고, 복원된 output에서 유효한 영역만 crop하도록 software reference model을 구성하였다.

3.1.2 Transform-domain Quantization-Aware Training

FP32 모델의 weight를 학습 이후 곧바로 INT4로 변환하는 PTQ를 적용한 결과, quantization error로 인해 accuracy가 감소하였다. 특히 본 설계에서 실제 multiplier에 입력되는 weight는 spatial weight g가 아니라 transform-domain weight U이므로, hardware에서 사용되는 U의 quantization error를 학습 과정에 직접 반영할 필요가 있다. 이를 위해 FP32 spatial master weight g를 유지하면서 매 forward 과정에서 U를 생성하고, 생성된 U에 signed INT4 fake quantization을 적용하는 Winograd-aware QAT를 구성하였다.

$$\hat{U} = Q_INT4(GgG^T) \tag{7}$$

여기서 Q_INT4는 signed INT4 quantization을 모사하는 fake-quantization operator이며, Winograd convolution의 forward 연산에는 FP32 U가 아니라 quantized weight $\hat{U}$가 사용된다.

Layer 사이에 저장되는 activation에는 hardware activation format과 동일한 unsigned INT8 fake quantization을 적용하였다.

$$\hat{a} = Q_UINT8(a) \tag{8}$$

FC weight에도 signed INT4 fake quantization을 적용하였다. Fake quantization의 round 연산에는 Straight-Through Estimator를 적용하여 gradient가 FP32 master parameter까지 전달되도록 하였다 [4]. 따라서 transform-domain weight U를 직접 학습하는 것이 아니라, quantized U를 사용한 forward 결과를 기반으로 spatial master weight g를 갱신한다.

전체 학습 및 parameter 생성 과정은 Algorithm 1과 같다. FP32 spatial model을 학습한 후 Winograd-aware QAT를 수행하고, 학습이 완료되면 integer bias와 requantization parameter를 생성한다. 생성된 parameter는 마지막으로 integer golden model을 이용하여 검증한다.

Algorithm 1 Winograd-aware QAT and Parameter Generation

Input: training dataset $\mathcal{D}$, FP32 spatial CNN parameters θ
Output: quantized weights, biases, and requantization parameters

Train the 3×3 spatial CNN in FP32.
Initialize the Winograd QAT model using θ.
for each training iteration **do**
  Quantize stored activations: $\hat{a} \leftarrow Q_{UINT8}(a)$
  Transform spatial weights: $U \leftarrow GgG^T$
  Quantize transformed weights: $\hat{U} \leftarrow Q_{INT4}(U)$
  Quantize FC weights: $\hat{W}_{FC} \leftarrow Q_{INT4}(W_{FC})$
  Perform forward propagation using $\hat{a}, \hat{U}, \hat{W}_{FC}$
  Update FP32 master parameters through the STE.
end for
Convert biases and generate requantization parameters.
Export and verify parameters using the integer golden model.

Algorithm 1. Winograd-aware QAT and Parameter Generation

학습 및 quantization 조건은 Table 3과 같다. Final fine-tuning 단계에서는 quantization scale의 변화를 제한하기 위해 observer를 고정하고 data augmentation을 적용하였다.

Stage	Item	Configuration
Common	Dataset	MNIST
FP32	Pretraining	SGD, 25 epochs
INT4 QAT	Initial fine-tuning	SGD, 8 epochs
INT4 QAT	Final fine-tuning	Adam, 10 epochs, augmentation
INT4 QAT	Quantization scale	Observers frozen during final fine-tuning
Numerical	Weights	Transformed U / FC, signed INT4
Numerical	Stored activation	Unsigned INT8
Training	Gradient approximation	Straight-Through Estimator
Export	Bias	Accumulator-domain signed INT16

Table 3. Winograd-aware QAT configuration

3.1.3 Accuracy 분석

FP32 3×3 spatial model은 MNIST test set 10,000장에 대해 97.03%의 accuracy를 기록하였다. 동일한 모델에 학습 과정 없이 W4A8 PTQ를 적용하면 accuracy가 95.21%로 감소하였다.

Transform-domain INT4 QAT를 적용한 결과, MNIST test set accuracy는 96.25%까지 회복되었다. 대회에서 제공한 1,000장 입력에서는 PTQ 모델이 94.00%를 기록한 반면, 최종 QAT 모델은 목표 accuracy인 97.00%를 달성하였다.

Model	Weight	MNIST (10,000)	Competition (1,000)
Spatial 3×3 FP32	FP32	97.03%	—
Winograd W4A8 PTQ	INT4	95.21%	94.00%
Winograd W4A8 QAT	INT4	96.25%	97.00%
Exported integer model	INT4	96.25%	97.00%

Table 4. Accuracy comparison of FP32, PTQ, and QAT models

이 결과는 spatial weight를 학습 이후 단순히 INT4로 변환하는 것보다, 실제 hardware에서 사용하는 transform-domain weight U의 quantization error를 학습 과정에 반영하는 것이 효과적임을 보여준다. Export 된 integer parameter를 다시 읽어 수행한 inference에서도 QAT checkpoint와 동일한 accuracy를 기록하였다.

3.2 Integer 연산 및 Numerical Format 검증

QAT는 quantization error를 학습 과정에 반영하지만, RTL의 모든 intermediate operation을 실제 integer arithmetic으로 수행하지는 않는다. 따라서 RTL과 동일한 연산 순서, rounding 및 saturation을 적용하는 bit-accurate integer golden model을 별도로 구성하였다.

Integer golden model은 Winograd input transform, element-wise multiplication, channel reduction, output transform, bias addition, ReLU, max-pooling 및 requantization을 RTL과 동일한 순서로 수행한다. FC layer의 multiplication과 accumulation 역시 integer arithmetic으로 모사하여 각 stage의 value range와 saturation 발생 횟수를 측정하였다.

최종 numerical format과 선택 근거는 Table 5와 같다. Input 및 stored activation은 대회 interface와 activation QAT 형식에 맞춰 UINT8을 사용하였다. Winograd U와 FC weight는 W4A8 QAT를 통해 INT4로 축소하였다. Input transform 결과 V는 전체 test set에서 −510~1020의 범위로 관측되어, 이를 saturation 없이 표현하기 위한 INT11을 적용하였다. Bias는 scale 변화와 구현 margin을 고려하여 INT16으로 저장하였다.

Data	Format	Rationale
Activation	**UINT8**	I/O interface and activation QAT; code range 0–255
U / FC weight	**INT4**	W4A8 QAT accuracy 96.25%
V	**INT11**	Observed −510 to 1020; minimum saturation-free width
Compute	**INT18**	INT14 saturation-free; INT18 retained as current RTL margin
Bias	**INT16**	Observed requirement ≤ 9 bit; scale and implementation margin
Requantization	**24-bit / S=24**	24-bit scale approximation; integer round-trip mismatch 0

Table 5. Numerical format과 bit-width 선택 근거

Product, channel reduction, output transform 및 FC accumulation은 동일한 compute datapath를 재사용한다. 최종 W4A8 모델에서 compute width에 따른 accuracy와 block별 saturation 발생 횟수를 측정한 결과를 Fig. 2에 나타내었다.

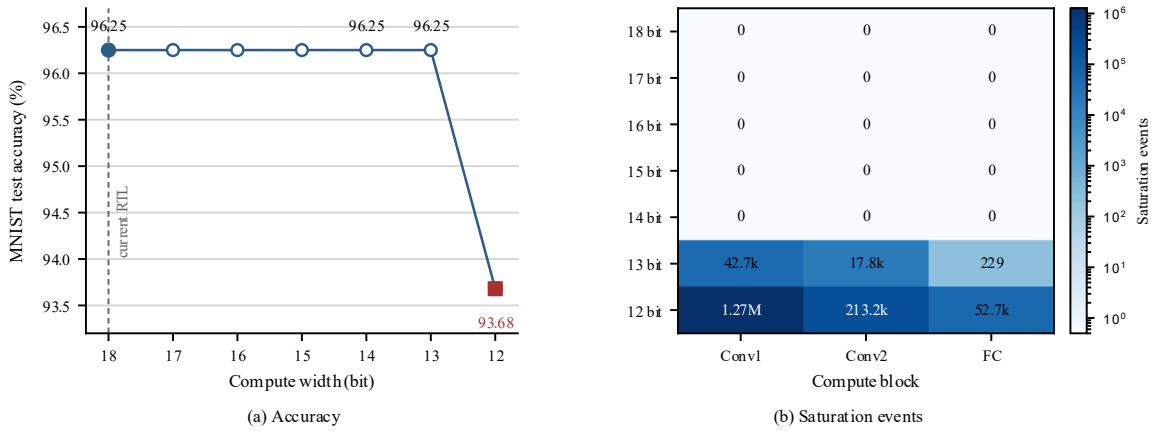


Fig. 2. Final W4A8 model compute-width sweep: (a) classification accuracy, (b) block별 saturation 발생 횟수

INT14까지는 MNIST test set 10,000장을 처리하는 동안 saturation이 발생하지 않았으며 96.25%의 accuracy가 유지되었다. INT13에서는 총 60,645회의 saturation이 발생했지만 accuracy는 유지되었다. INT12에서는 약 153만 회의 saturation과 함께 accuracy가 93.68%로 감소하였다.

관측된 test set에서는 INT14가 saturation-free 최소 폭이지만, 현재 RTL에서는 input distribution 변화와 accumulation에 대한 numerical margin을 확보하기 위해 INT18을 사용하였다. INT14 또는 INT15 적용 여부는 향후 synthesis를 통해 area, power 및 Fmax를 비교한 후 결정한다.

학습 완료 후 transform-domain INT4 weight, INT16 bias 및 requantization parameter를 RTL 입력 형식으로 export하였다. Export 파일을 다시 읽어 수행한 integer inference는 QAT checkpoint와 prediction 및 integer logit이 모두 일치했으며, TXT와 HEX 사이의 mismatch도 0이었다. 최종 integer model은 MNIST test set에서 96.25%, 대회 입력 1,000장에서 97.00%의 accuracy를 기록하였다.

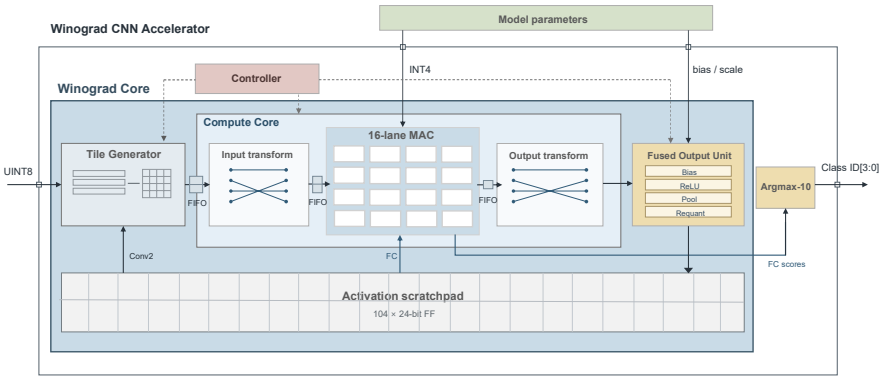
3.3 Hardware Architecture 및 Dataflow

Fig. 1은 본 설계의 single-core Winograd CNN accelerator architecture를 나타낸다. 전체 hardware는 Controller, Winograd Core, Model Parameter Interface 및 Argmax-10으로 구성된다. Winograd Core에는 Tile Generator, Compute Core, Fused Output Unit 및 Activation Scratchpad가 포함된다. Compute Core 내부에는 Input Transform, 16-lane MAC 및 Output Transform이 배치되며, 각 pipeline boundary는 elastic FIFO로 연결된다.

Conv1에서는 외부 UINT8 pixel stream으로부터 4×4 input tile을 생성한다. Input Transform에서 transform-domain activation V를 계산한 후, 16-lane MAC에서 transform-domain weight U와 element-wise multiplication을 수행한다. Output Transform에서 복원된 2×2 output tile은 Fused Output Unit에서 bias addition, ReLU, max pooling 및 requantization을 거쳐 다음 layer의 activation으로 저장된다.

Conv2에서는 Conv1 activation을 동일한 Winograd Core에 다시 공급한다. 세 input channel에 대한 multiplication result는 MAC 내부에서 channel reduction되며, 마지막 channel까지 누적된 결과만 Output Transform으로 전달된다. Conv2 output은 이미 소비된 Conv1 activation의 scratchpad 영역에 저장하여 별도의 output buffer를 사용하지 않는다.

Conv2가 완료되면 Controller는 16-lane MAC을 FC mode로 전환한다. Lane 0–9는 각각 하나의 class를 담당하여 10개의 FC score를 병렬로 누적하고, lane 10–15는 비활성화된다. 최종 score는 Argmax-10으로 전달되며 가장 큰 값의 index가 4-bit Class ID로 출력된다. 이와 같이 하나의 Winograd Core를 Conv1, Conv2 및 FC에서 시간적으로 재사용하여 layer별 연산기의 중복을 제거하였다.



(a) Single-core Winograd CNN accelerator

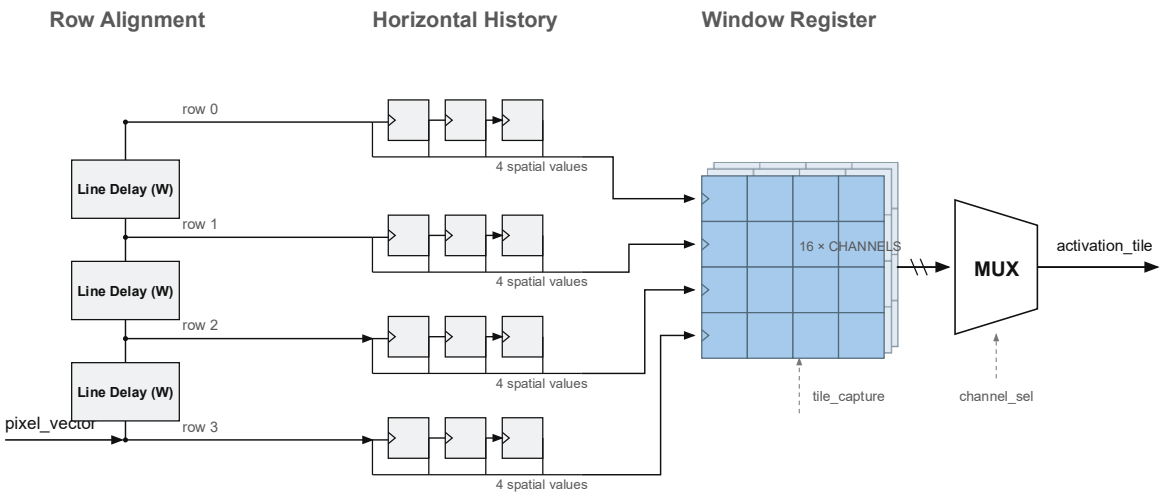
Fig. 1. Single-core Winograd CNN accelerator architecture.

3.4 Winograd Core Microarchitecture

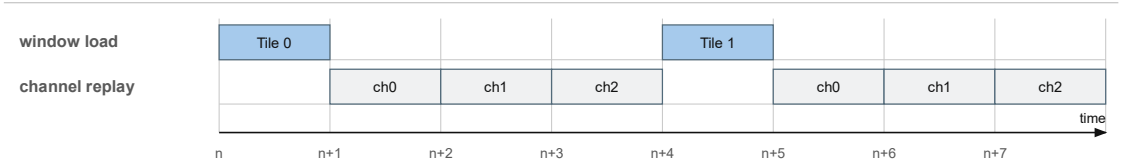
3.4.1 Tile Generator

Tile Generator는 raster-scan 순서로 입력되는 activation을 line buffer와 horizontal shift register에 저장하고, $F(2 \times 2,\ 3 \times 3)$ 연산에 필요한 4×4 input tile을 생성한다. 고정된 data movement를 register shift로 구현하여 임의 주소를 선택하는 대규모 multiplexer 구조를 사용하지 않았다.

Conv1에서는 외부 pixel stream이 직접 입력되고, Conv2에서는 Activation Scratchpad에서 읽은 세 channel의 packed activation이 입력된다. Channel selector는 실행 중인 input channel에 해당하는 byte를 선택하며, line buffer와 shift register에 필요한 activation이 채워진 시점에 하나의 4×4 tile을 출력한다.



(a) Sliding-window generation datapath



(b) Three-channel replay schedule

Fig. 3. Microarchitecture of the Winograd Tile Generator.

3.4.2 Input Transform

Input Transform은 4×4 spatial tile d에 $B^T dB$ 연산을 적용하여 transform-domain activation V를 생성한다. 본 설계에서 사용하는 F(2×2, 3×3)의 input transform matrix B^T는 다음과 같다 [2].

$$B^T = \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix} \tag{9}$$

Matrix의 모든 coefficient가 0과 ±1이므로 constant multiplication은 wire selection, addition 및 subtraction으로 치환할 수 있다. 하나의 4-element vector $[d_0, d_1, d_2, d_3]$에 대한 1-D transform은 다음과 같다.

$$t_0 = d_0 - d_2, \qquad t_1 = d_1 + d_2, \qquad t_2 = d_2 - d_1, \qquad t_3 = d_1 - d_3$$

따라서 Input Transform에는 일반적인 multiplier가 필요하지 않으며 네 개의 add/subtract datapath로 구현할 수 있다. 2-D transform은 column 방향의 1-D transform을 수행하여 intermediate result를 TEMP Register Bank에 저장한 후, 동일한 연산을 row 방향으로 적용하는 2-stage pipeline으로 구성하였다. 두 transform stage 사이에는 register boundary를 배치하여 combinational path를 분리하였다.

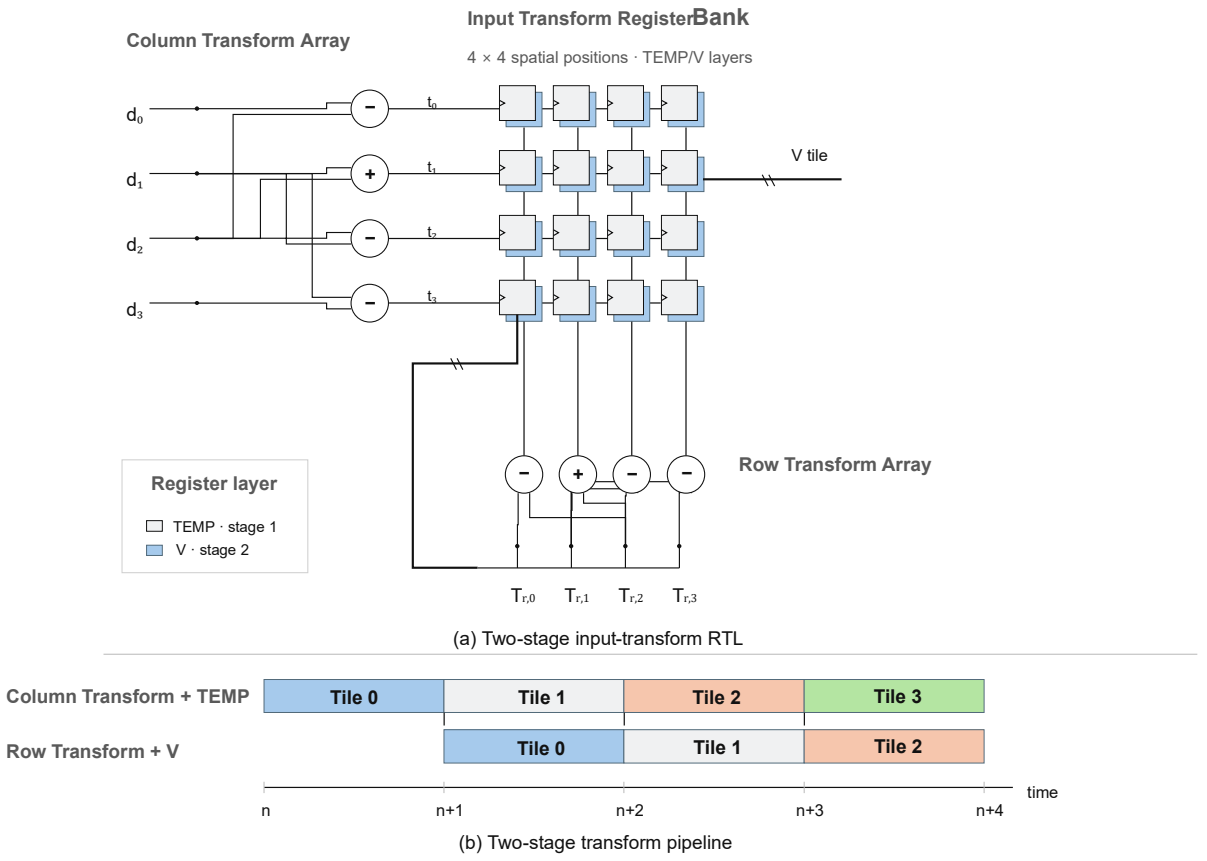


Fig. 4. Two-stage Input Transform microarchitecture.

3.4.3 Reconfigurable 16-lane MAC

16-lane MAC은 Winograd convolution과 FC layer에서 공유된다. Convolution mode에서 각 lane은 4×4 Winograd domain의 한 position을 담당한다. 각 lane은 INT11 activation V와 INT4 weight U를 곱하고, input channel에 따라 결과를 INT18 accumulator에 누적한다.

Conv1은 하나의 input channel을 사용하므로 별도의 channel accumulation 없이 결과를 출력한다. Conv2에서는 세 input channel의 multiplication result를 같은 accumulator에 순차적으로 더하고, 마지막 channel에서만 누적 결과를 RESULT register에 기록한다.

FC mode에서는 동일한 UINT8 activation을 lane 0–9에 공급하고 각 class에 대응하는 INT4 weight를 곱한다. 각 lane은 75개의 feature에 대한 결과를 시간적으로 누적하며, 마지막 cycle에 해당 class의 FC bias를 더한다. 사용하지 않는 lane 10–15는 비활성화된다.

Weight가 0이거나 lane이 비활성 상태인 경우에는 multiplier 입력을 0으로 고정하는 operand isolation을 적용하였다. 연산 cycle과 pipeline timing은 유지하면서 불필요한 data switching이 multiplier 내부로 전파되는 것을 억제하였다.

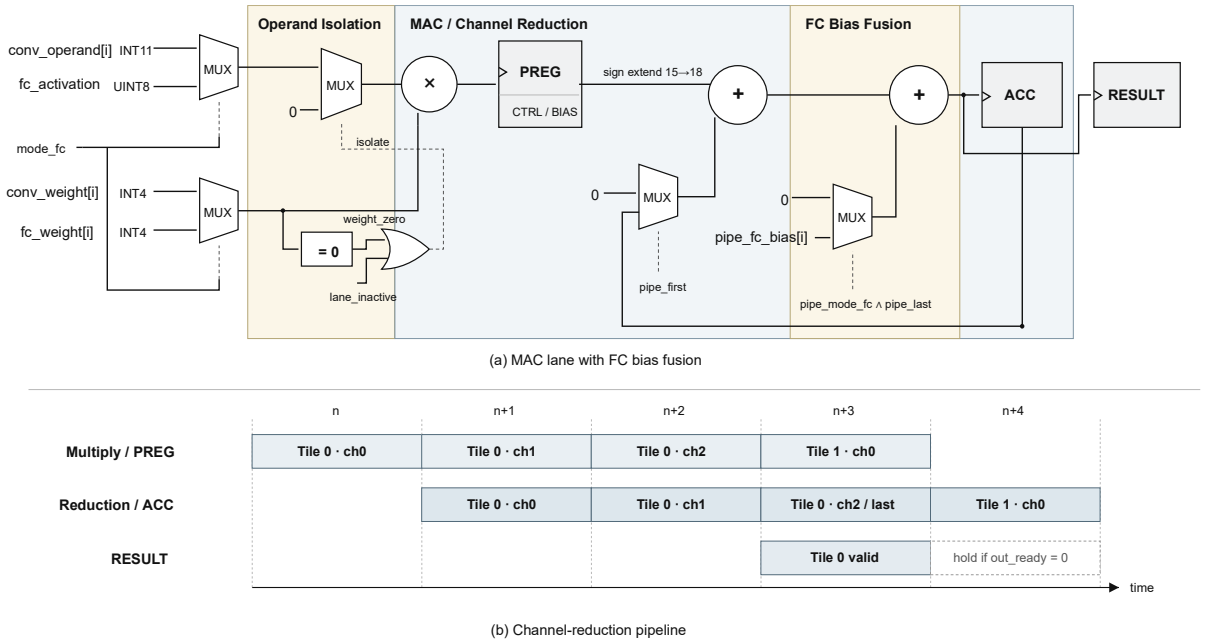


Fig. 5. Reconfigurable MAC Lane with FC bias fusion and pipeline schedule.

3.4.4 Output Transform

Output Transform은 channel reduction이 완료된 4×4 intermediate result M에 $A^{\mathrm{T}}MA$ 연산을 적용하여 2×2 spatial output tile Y를 생성한다. 사용된 output transform matrix A^{T}는 다음과 같다 [2].

$$A^T = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{bmatrix} \tag{10}$$

A^T 역시 coefficient가 0과 ±1로만 구성된다. 하나의 4-element vector $[m_0, m_1, m_2, m_3]$에 대한 1-D output transform은 다음과 같다.

$$l_0 = m_0 + m_1 + m_2, \qquad l_1 = m_1 - m_2 - m_3$$

따라서 Output Transform도 multiplier 없이 add/subtract unit만으로 구현할 수 있다. 첫 번째 stage에서는 $A^T M$을 계산하여 2×4 intermediate result를 Register Bank에 저장하고, 두 번째 stage에서는 오른쪽 matrix A를 적용하여 최종 2×2 output tile을 생성한다. 각 addition과 subtraction에는 INT18 saturation을 적용하여 hardware와 integer inference model의 numerical behavior를 일치시켰다.

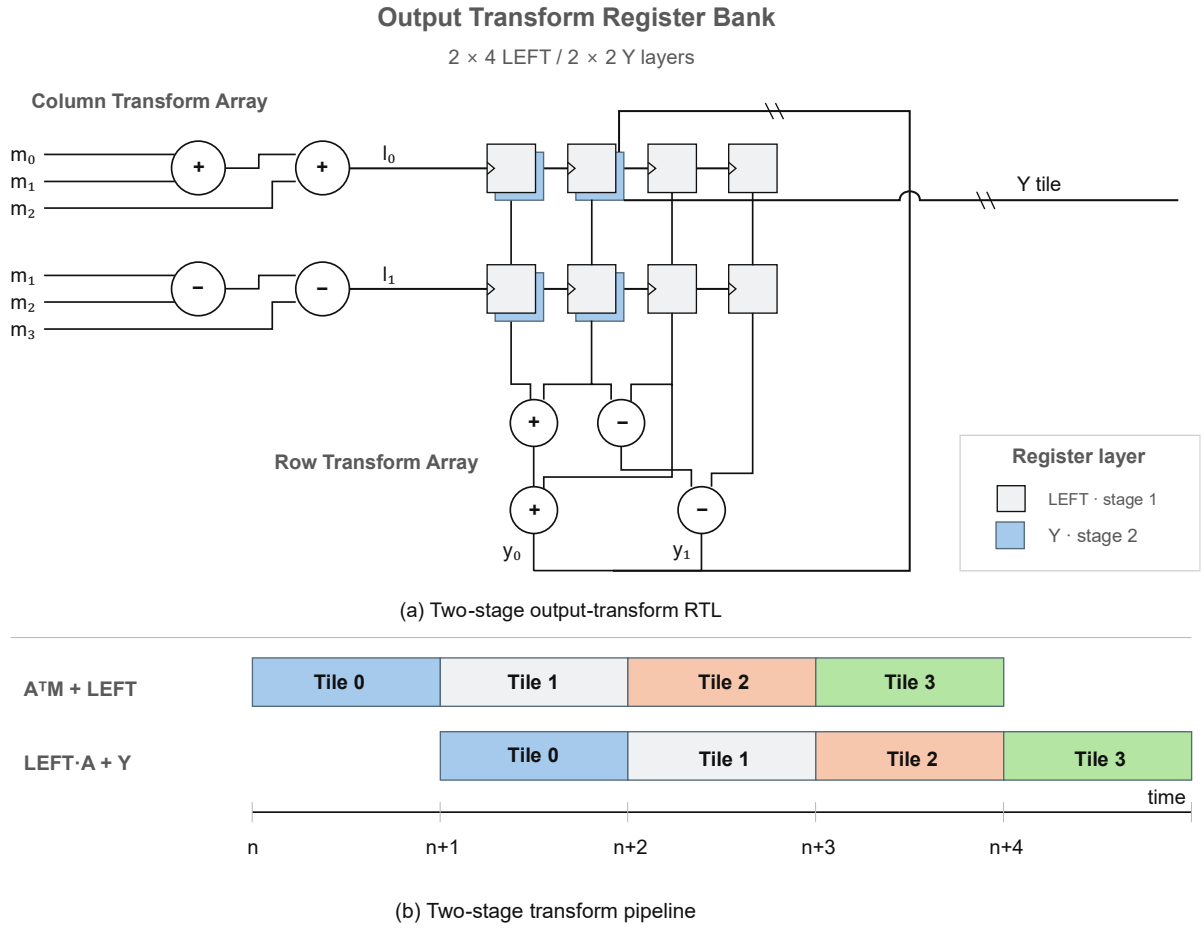


Fig. 6. Two-stage Output Transform microarchitecture.

3.4.5 Fused Output Unit

Output Transform에서 생성된 2×2 tile은 별도의 intermediate buffer에 기록되지 않고 Fused Output Unit으로 직접 전달된다. 이 블록은 convolution bias addition, ReLU, 2×2 max pooling 및 requantization을 tile 단위로 연속 수행한다.

일반적인 layer-by-layer 구조에서는 convolution result를 저장한 후 nonlinear operation을 위해 다시 읽어야 한다. 반면 본 설계는 output tile이 생성되는 시점에 후속 operation을 결합하여 intermediate activation의 write/read를 제거하였다 [3]. 최종 UINT8 activation만 Activation Scratchpad에 기록한다.

Requantization은 accumulator-domain result에 layer별 integer multiplier를 곱한 후 고정된 shift를 적용한다. 음수는 ReLU에 의해 0으로 제한하고, UINT8 범위를 초과하는 값은 255로 saturation한다.

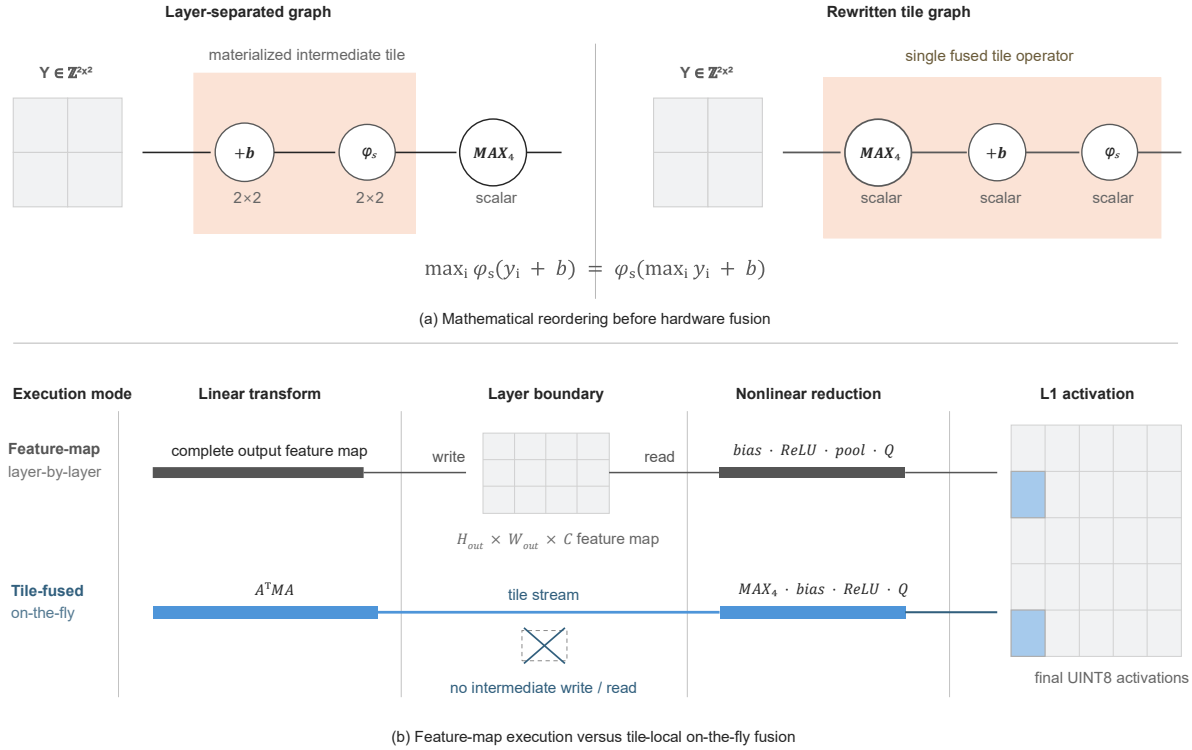


Fig. 7. Tile-level operation flow of the Fused Output Unit.

3.5 Buffer 및 Activation Storage 최적화

3.5.1 Elastic FIFO

서로 다른 latency와 처리율을 갖는 pipeline block을 분리하기 위해 valid/ready handshake 기반의 elastic communication channel을 적용하였다 [6]. 각 pipeline boundary에는 FF 기반 synchronous FIFO를 배치하여 downstream stall이 발생한 경우에도 전송 중인 data를 보존하도록 하였다.

Producer와 consumer가 모두 준비된 cycle에만 data transfer가 발생하며, consumer가 stall되면 FIFO가 해당 data를 유지한다. 동시에 read와 write가 발생할 수 있으므로 pipeline throughput을 유지하면서 block 사이의 순간적인 처리율 차이를 흡수할 수 있다.

FIFO의 storage width와 depth는 각 boundary의 data format과 RTL simulation 결과에 따라 독립적으로 설정하였다. Conv1 Tile FIFO에는 4×4 activation tile, V-replay FIFO에는 transform-domain activation, M FIFO에는 channel reduction이 완료된 intermediate result가 저장된다.

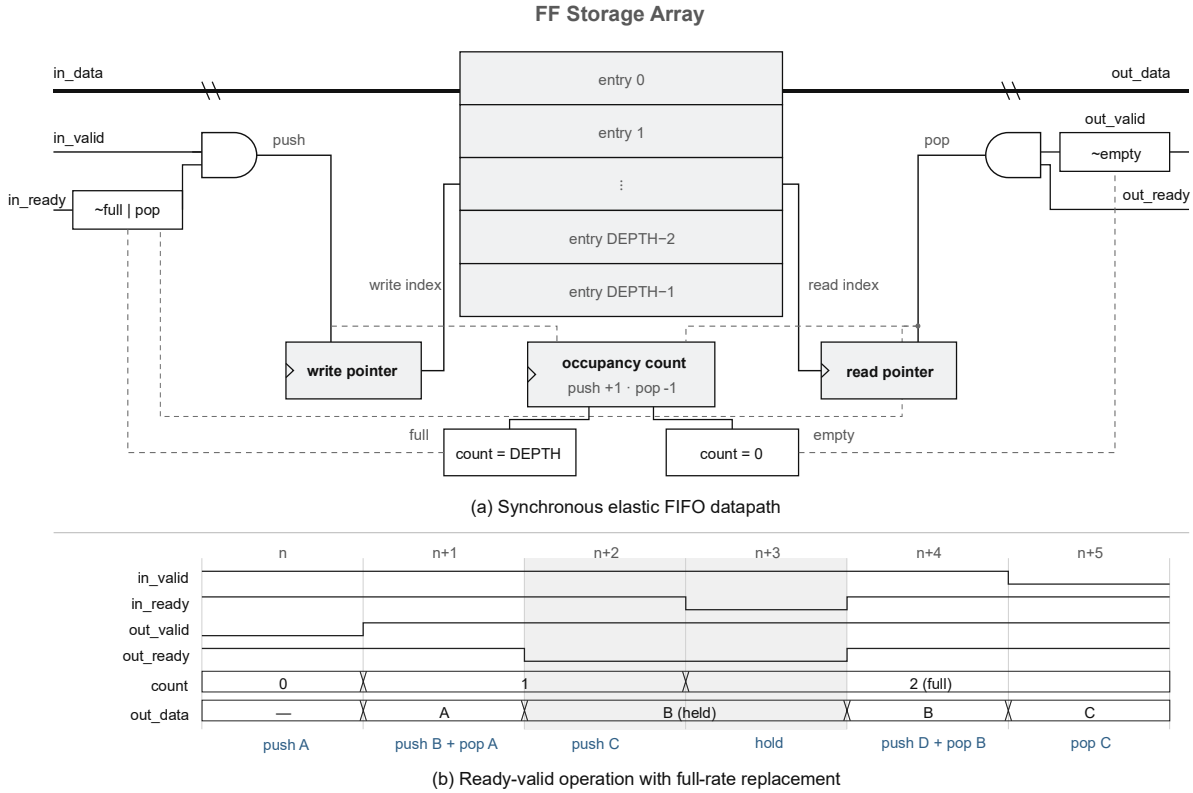


Fig. 8. Elastic FIFO microarchitecture and valid-ready timing.

3.5.2 FIFO Depth 최적화

FIFO depth가 지나치게 크면 사용되지 않는 FF가 증가하고, 반대로 너무 작으면 internal stall이 외부 pixel interface까지 전파될 수 있다. 이에 따라 Conv1 Tile FIFO의 depth를 1, 2, 4, 8로 변경하며 maximum occupancy, input backpressure 및 1,000-image accuracy를 측정하였다.

Depth 1에서는 core stall을 흡수하지 못해 영상당 36 cycle의 input backpressure가 발생하였다. 대회 input interface는 ready 신호에 따라 pixel 전송을 중단하지 않으므로 pixel alignment가 깨졌으며, accuracy는 9%까지 감소하였다. 반면 Depth 2 이상에서는 maximum occupancy가 2를 넘지 않았고 input stall 없이 97% accuracy를 유지하였다. 이에 따라 Tile FIFO를 기존 Depth 8에서 Depth 2로 축소하였다.

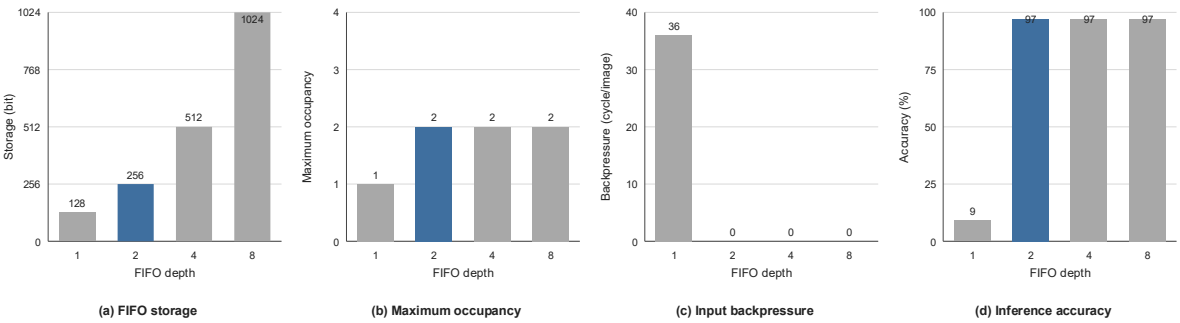


Fig. 9. Conv1 Tile FIFO depth sweep results

Fig. 10은 처음으로 core stall이 발생하는 구간의 cycle trace를 나타낸다. Depth 1에서는 FIFO가 가득 차면서 image_ready가 deassert되지만, Depth 2에서는 두 번째 entry가 tile을 보존하여 외부 pixel stream이 중단되지 않는다.

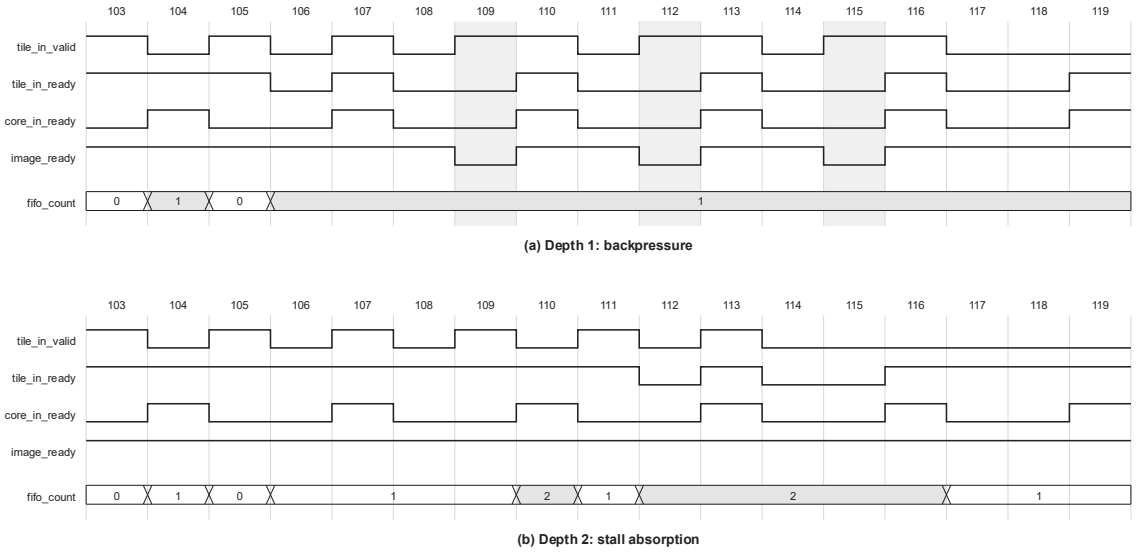


Fig. 10. FIFO occupancy and input backpressure according to FIFO depth.

V-replay FIFO와 M FIFO도 동일한 방식으로 depth를 변경하여 검증하였다. V-replay FIFO는 Depth 1에서 input loss가 발생하였으므로 Depth 2를 유지하였다. M FIFO는 maximum occupancy가 1이었으며, Depth 1에서도 기능, latency 및 accuracy가 동일하였다.

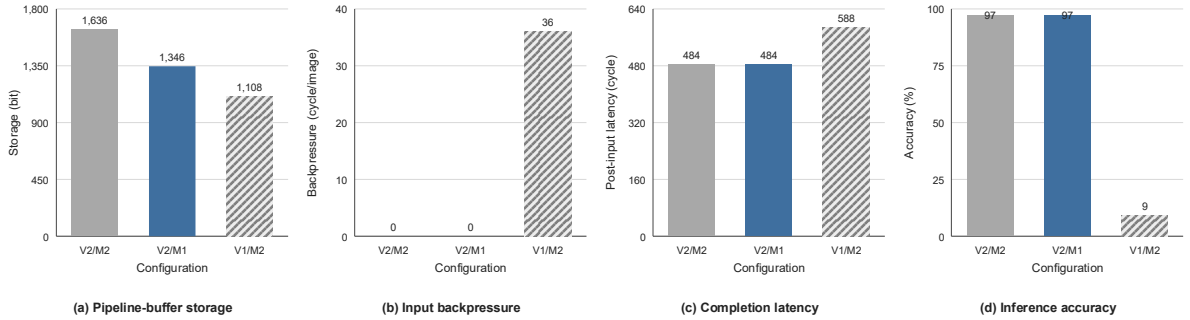


Fig. 11. Pipeline buffer depth optimization results.

최종 pipeline buffer configuration은 Table 5와 같다.

Buffer	Pipeline boundary	Maximum occupancy	Selected depth
Tile FIFO	Tile Generator → Input Transform	2	**2**
V-replay FIFO	Input Transform → MAC Array	2	**2**
M FIFO	MAC Array → Output Transform	1	**1**

Table 5. Selected pipeline buffer depths

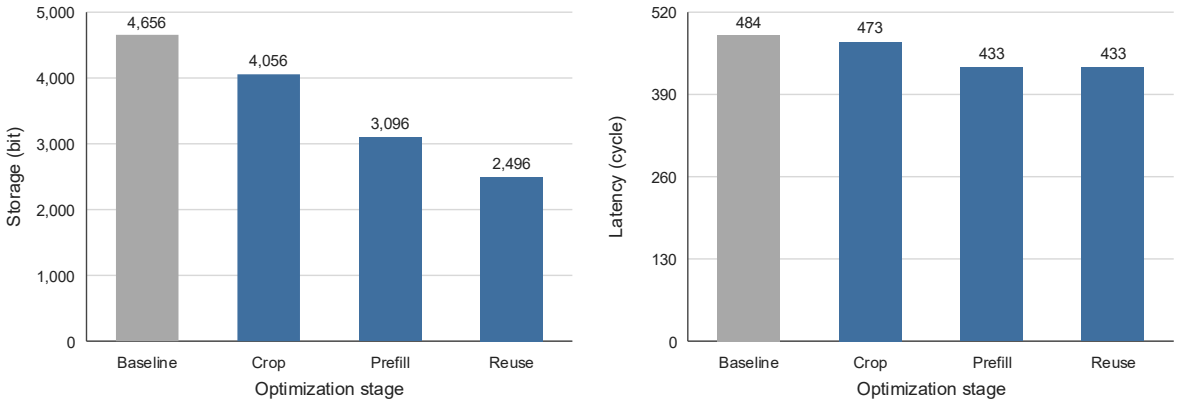
Tile FIFO를 Depth 8에서 2로 축소함으로써 해당 FIFO의 storage는 1,024 bit에서 256 bit로 75% 감소하였다. 동일한 synthesis condition에서 total cell area는 5.97%, FF 수는 6.67%, vectorless power는 6.62% 감소하였다. 최종 configuration에서도 1,000-image RTL mismatch 0과 accuracy 97%를 유지하였다.

3.5.3 Lifetime-aware Activation Storage

초기 구조에서는 13×13 Conv1 output 전체와 $5 \times 5 \times 3$ Conv2 output을 별도의 buffer에 저장하였다. 그러나 Conv2의 실제 reference 영역과 activation lifetime을 분석하면 두 data를 동시에 유지할 필요가 없다.

먼저 Conv2에서 참조하지 않는 Conv1 output의 마지막 row와 column을 제거하였다. 이후 첫 번째 4×4 Conv2 window를 구성하는 40개 vector는 Conv1 실행 중 Tile Generator에 직접 prefill하고, 나머지 104개 vector만 Activation Scratchpad에 저장하였다.

Conv2가 scratchpad의 activation을 순차적으로 읽으면 소비가 완료된 address가 발생한다. 본 설계는 이 영역의 address 0–24에 Conv2 output을 덮어쓴다. Conv2 output은 동일 spatial position의 세 channel을 하나의 24-bit word로 packing하며, FC layer에서는 byte lane을 순차적으로 선택하여 channel-major 순서를 복원한다. 이에 따라 별도의 FC feature buffer를 제거하였다.



(a) Inter-layer storage **(b) Post-input latency**

Fig. 12. Lifetime-aware activation storage optimization.

Inter-layer activation storage는 초기 4,656 bit에서 최종 2,496 bit로 46.39% 감소하였다. 또한 마지막 pixel 입력 이후의 처리 cycle은 484 cycle에서 433 cycle로 10.54% 감소하였다. 각 최적화 단계에서 RTL mismatch 0, accuracy 97% 및 input backpressure 0 cycle을 유지하였다.

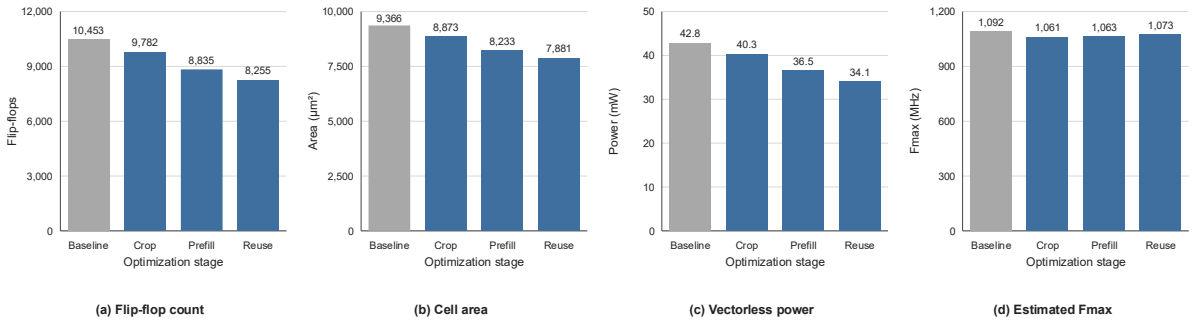


Fig. 13. Post-synthesis PPA improvement from activation storage optimization.

최종 consumed-region reuse 구조는 초기 169-entry 구조 대비 FF 수를 21.03%, total cell area를 15.86%, sequential area를 20.08% 감소시켰다. Vectorless power 역시 동일한 synthesis condition에서 20.33% 감소하였다.

3.6 RTL 검증 및 합성 결과

3.6.1 전체 RTL 기능 검증

학습을 통해 생성한 INT4 weight, integer bias, requantization parameter 및 MNIST test vector를 RTL 검증 환경에 적용하였다. 검증 범위에는 Tile Generator, Winograd Input/Output Transform, 16-lane MAC, Fused Output Unit, FC 연산 및 Argmax-10까지의 전체 inference datapath를 포함하였다.

10,000개의 MNIST test image 중 대회 평가 규모에 해당하는 1,000개 image를 입력하고, 각 image에 대한 RTL 출력과 PyTorch integer golden model의 출력을 비교하였다. 검증 결과 RTL과 golden model 사이의 mismatch는 발생하지 않았으며, 1,000개 중 970개를 올바르게 분류하여 97.0% accuracy를 확인하였다. 또한 전체 검증 과정에서 input backpressure가 발생하지 않아 외부에서 매 cycle 입력되는 pixel stream을 손실 없이 처리하였다.

한 image의 입력에는 784 cycle이 필요하며, 마지막 pixel 입력 이후 최종 Class ID가 출력되기까지 평균 433 cycle이 추가로 소요되었다. 따라서 전체 inference latency는 평균 1,217 cycle이며, 비교 표에서는 약 1,220 cycle로 정리하였다.

```
## run all
Progress: 100/1000, correct=97
Progress: 200/1000, correct=194
Progress: 300/1000, correct=291
Progress: 400/1000, correct=388
Progress: 500/1000, correct=485
Progress: 600/1000, correct=582
Progress: 700/1000, correct=679
Progress: 800/1000, correct=776
Progress: 900/1000, correct=873
Progress: 1000/1000, correct=970
RTL mismatches: 0
Correct: 970/1000
Accuracy: 97.0%
```

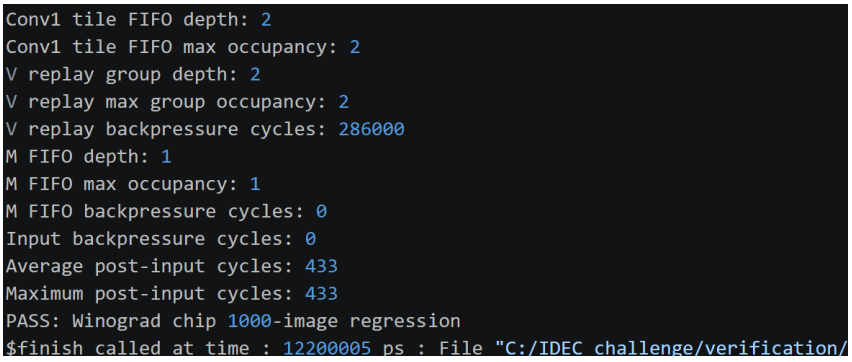


Fig. 11(a) 1,000-image RTL regression 결과

3.6.2 합성 및 timing 분석

제안한 accelerator는 ASAP7 standard-cell library를 이용하여 합성하였다. 목표 operating frequency는 1 GHz로 설정하였으며, STA 결과 minimum clock period는 847.31 ps, estimated Fmax는 1.180 GHz로 산출되었다. 따라서 본 설계는 1 GHz 동작 조건에서 timing margin을 확보한다.

Post-synthesis total cell area는 7,953.89 μm²로, 이를 환산하면 약 0.007954 mm²이다. 이 중 sequential cell area는 2,540.48 μm²로 전체 면적의 31.94%를 차지한다. Sequential cell의 비중은 line buffer, pipeline register 및 lifetime-aware activation scratchpad를 FF register array로 구현한 결과이다.

```
clk period_min = 847.31 fmax = 1180.21
Group                  Internal    Switching      Leakage        Total
                         Power        Power        Power        Power (Watts)
----------------------------------------------------------------------------
Sequential           1.269724e-02 1.181279e-04 1.351116e-06 1.281672e-02  77.2%
Combinational        1.626915e-03 2.148846e-03 3.909282e-06 3.779671e-03  22.8%
Clock                0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00   0.0%
Macro                0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00   0.0%
Pad                  0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00   0.0%
----------------------------------------------------------------------------
Total                1.432414e-02 2.266833e-03 5.260363e-06 1.659623e-02 100.0%
                         86.3%        13.7%         0.0%
```

[Fig. 11(b) STA 및 activity-based power report]

```
========================================================================
Post synthesis report_power
------------------------------------------------------------------------
Group                 Internal  Switching    Leakage      Total
                        Power      Power      Power      Power (Watts)
----------------------------------------------------------------
Sequential            1.18e-02   6.29e-06   1.09e-06   1.18e-02  86.7%
Combinational         9.56e-04   8.44e-04   4.41e-06   1.80e-03  13.3%
Clock                 0.00e+00   0.00e+00   0.00e+00   0.00e+00   0.0%
Macro                 0.00e+00   0.00e+00   0.00e+00   0.00e+00   0.0%
Pad                   0.00e+00   0.00e+00   0.00e+00   0.00e+00   0.0%
----------------------------------------------------------------
Total                 1.27e-02   8.50e-04   5.50e-06   1.36e-02 100.0%
                        93.7%       6.3%       0.0%
```

[Fig. 11(c) Post-synthesis cell area report]

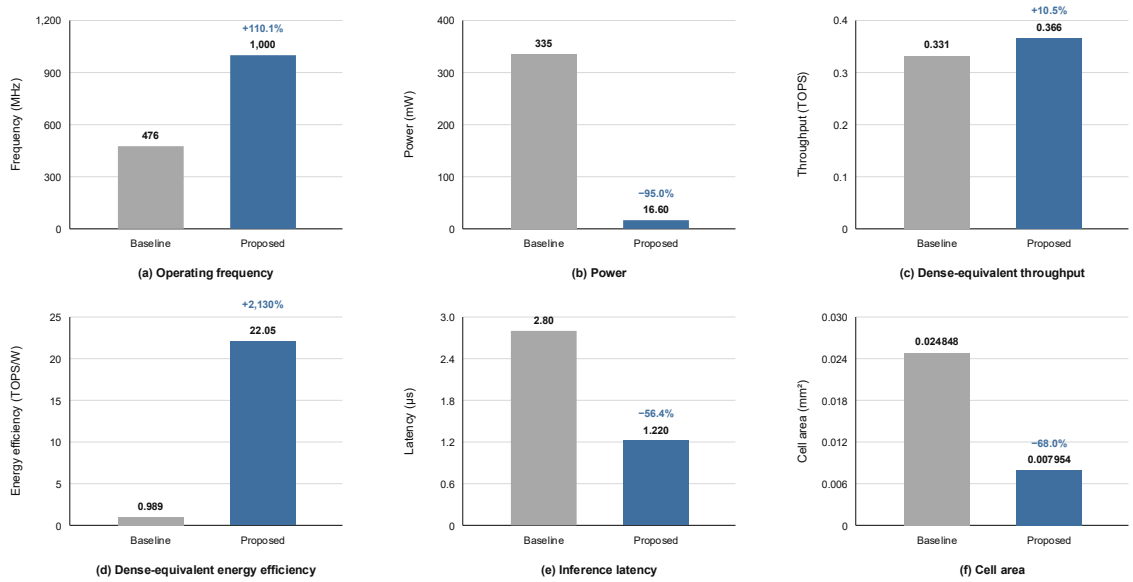


Fig. 11. RTL verification and post-synthesis results: (a) 1,000-image RTL regression, (b) STA and activity-based power, and (c) cell area

3.6.3 Activity-based power 분석

실제 RTL simulation에서 생성한 switching activity를 합성 netlist에 annotation하여 power를 분석하였다. 분석 결과 1 GHz 동작 조건에서 total power는 16.5962 mW로 산출되었다.

Sequential logic의 power는 12.8167 mW로 전체의 77.2%이며, combinational logic은 3.7797 mW로 22.8%를 차지하였다. Sequential power의 비중이 높은 것은 activation scratchpad와 pipeline register를 SRAM macro가 아닌 FF 기반 구조로 구현했기 때문이다. 반면 combinational logic에는 Winograd transform, 16-lane MAC 및 Fused Output Unit이 포함된다.

최종 성능 비교에는 임의의 toggle rate를 사용하는 vectorless power가 아니라, 실제 inference 동작으로부터 얻은 activity-based power를 사용하였다.

3.6.4 Dense-equivalent 연산 성능 산정

기존 설계와 동일한 조건에서 연산 성능을 비교하기 위해 대회 수상작 보고서에서 사용한 dense-equivalent 연산량 산정 방식을 적용하였다. MAC 한 회는 multiplication과 addition의 두 개 operation으로 계산하며, throughput과 energy efficiency는 다음과 같이 정의하였다.

$$TOPS = (2 \times N_MAC \times f) / 10^{12} \tag{11}$$

$$TOPS/W = TOPS / P \tag{12}$$

여기서 N_MAC은 CNN model의 dense-equivalent MAC 수, f 는 operating frequency, P 는 power consumption이다.

Baseline model의 dense-equivalent MAC 수는 다음과 같다.

$$N_MAC, baseline = (5 \times 5 \times 1 \times 3) + (5 \times 5 \times 3 \times 3) + 48 = 75 + 225 + 48 = 348$$

따라서 476 MHz에서 baseline의 throughput은 0.331 TOPS이며, 335 mW를 적용한 energy efficiency는 0.989 TOPS/W이다.

제안한 model의 dense-equivalent MAC 수는 다음과 같다.

$$N_MAC,proposed = (3 \times 3 \times 1 \times 3) + (3 \times 3 \times 3 \times 3) + 75 = 27 + 81 + 75 = 183$$

이에 따라 1 GHz에서 제안한 accelerator의 dense-equivalent throughput은 0.366 TOPS이다. Activity-based power인 16.5962 mW를 적용하면 energy efficiency는 22.05 TOPS/W이다.

이 값은 물리적으로 배치된 multiplier 수만으로 산출한 peak throughput이 아니라, 동일한 CNN inference를 direct convolution 기준의 dense operation으로 환산한 수치이다.

3.6.5 Baseline 대비 성능 비교

Table 6은 대회 baseline accelerator와 제안한 accelerator의 주요 성능을 비교한 결과이다. 제안한 설계는 baseline과 동일한 97.0% accuracy를 유지하면서 operating frequency를 476 MHz에서 1 GHz로 향상하였다.

전체 inference cycle은 1,335 cycle에서 약 1,220 cycle로 8.6% 감소하였다. 주파수 향상과 cycle 감소가 함께 반영되면서 inference latency는 2.80 μs에서 1.22 μs로 56.4% 감소하였다.

Cell area는 0.024848 mm²에서 0.007954 mm²로 68.0% 감소하였으며, power는 335 mW에서 16.60 mW로 95.0% 감소하였다. Dense-equivalent throughput은 0.331 TOPS에서 0.366 TOPS로 10.5% 향상되었다. 특히 TOPS/W는 0.989에서 22.05로 약 22.3배 향상되었다.

Metric	Baseline	Proposed	Change
Accuracy	97.0%	**97.0%**	Maintained
Operating frequency	476 MHz	**1 GHz**	+110.1%
Power	335 mW	**16.60 mW**	−95.0%
Dense-equivalent throughput	0.331 TOPS	**0.366 TOPS**	+10.5%
Dense-equivalent energy efficiency	0.989 TOPS/W	**22.05 TOPS/W**	22.3×
Cycles per image	1,335	**1,220**	−8.6%
Inference latency	2.80 μs	**1.220 μs**	−56.4%
Cell area	0.024848 mm²	**0.007954 mm²**	−68.0%

Table 6. Performance comparison between the baseline and proposed accelerator

4. 앞으로의 진행 계획

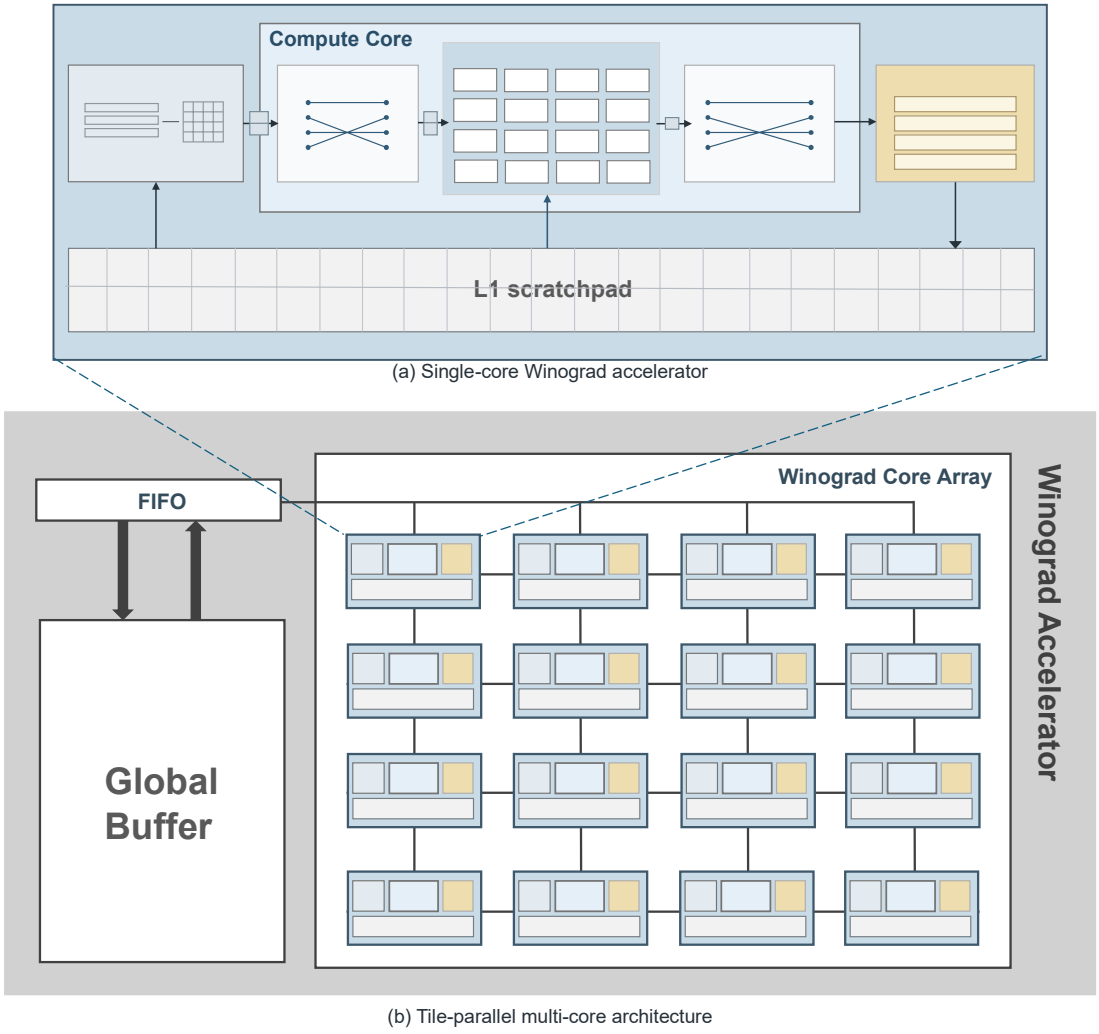


Fig. 13. Extension from a single-core to a tile-parallel multi-core Winograd accelerator: (a) single-core architecture and (b) multi-core architecture.

본 과제에서는 하나의 Winograd Core를 재사용하여 CNN inference를 수행하는 single-core accelerator를 구현하였다. 향후 자유주제에서는 Fig. 13과 같이 동일한 core를 복수로 배치하고, input tensor를 tile 단위로 분산하여 병렬 처리하는 multi-core architecture로 확장할 계획이다.

각 core는 local scratchpad를 기반으로 독립적으로 tile을 처리하며, global buffer와 scheduler가 core별 workload와 data movement를 관리한다. 이를 기반으로 VGG 및 ResNet 계열의 대규모 CNN workload를 대상으로 3×3 convolution의 병렬 처리와 core 수에 따른 scalability를 검증하고자 한다. 추가로 ResNet의 1×1 convolution과 residual connection을 지원하기 위한 bypass datapath 및 scheduling 구조를 설계할 예정이다.

5. 참고문헌

[1] Y.-H. Chen, T. Krishna, J. Emer, and V. Sze, "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.

[2] A. Lavin and S. Gray, "Fast Algorithms for Convolutional Neural Networks," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 4013–4021, 2016.

[3] M. Alwani, H. Chen, M. Ferdman, and P. Milder, "Fused-Layer CNN Accelerators," *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture*, pp. 1–12, 2016.

[4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2704–2713, 2018.

[5] J. Fernandez-Marques, P. N. Whatmough, A. Mundy, and M. Mattina, "Searching for Winograd-aware Quantized Networks," *Proceedings of Machine Learning and Systems*, vol. 2, pp. 14–29, 2020.

[6] J. Cortadella, M. Kishinevsky, and B. Grundmann, "Synthesis of Synchronous Elastic Architectures," *Proceedings of the 43rd Design Automation Conference*, pp. 657–662, 2006.